

Data Mining and Decision Support

Project Report:

NBA Database Prediction

Team 5:

Daulet Bulegenov

Daniyar Kakimbekov

Nurbek Malikov

Spring 2025,

Nazarbayev University

Dataset Selection

For this group assignment, we wanted to choose a relatively large and complicated dataset with many variables that we can use for both regression and classification tasks. The dataset should also be diverse enough for us to reuse it for the second group assignment. After searching the web, we decided to choose the NBA Database from Kaggle.

The NBA Database is a historical database of all of the games that were played in the NBA starting from the inception of the league in the 1946-1947 season and after the merger with ABA in the 1976 offseason.

Data Understanding

Initially, the dataset is stored in separate normalized tables with game, player, team, and season details as an SQLite database. We combined these separate tables into one complete dataset and then cleaned the data.

An SQL Query was used as a preprocessing method. The query aggregates, joins, and groups performance metrics for NBA games, teams, officials, and matchups from all seasons into a single comprehensive dataset for analysis. As a result, we got game information, home team metrics, away team metrics, officials' metrics, game-level metrics, and matchup metrics with 23466 rows and 43 features.

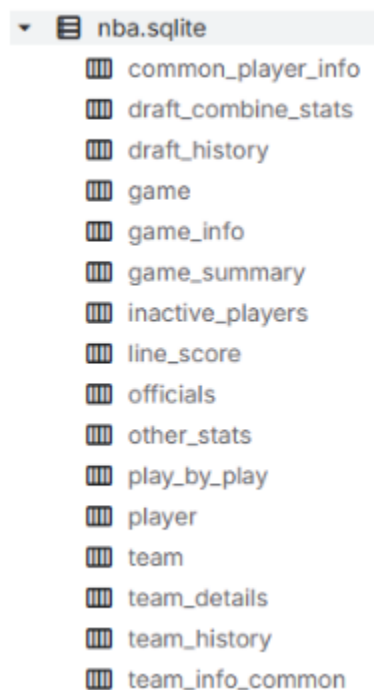


Figure 1. Dataset stored as normalized tables in SQLite database.

It turned out that the dataset didn't contain any missing values as a result of the SQL query, so no data cleaning was required. However, it was important to conduct exploratory data analysis to generate visualizations and summary statistics to identify distributions and trends across features.

Below is a histogram plot (Figure 2) with kernel density estimation constructed for the target variable. As seen this plot resembles the normal distribution with some skewness to the right. Most games fall between roughly 150 and 260 total points, with a few extreme values beyond 300.

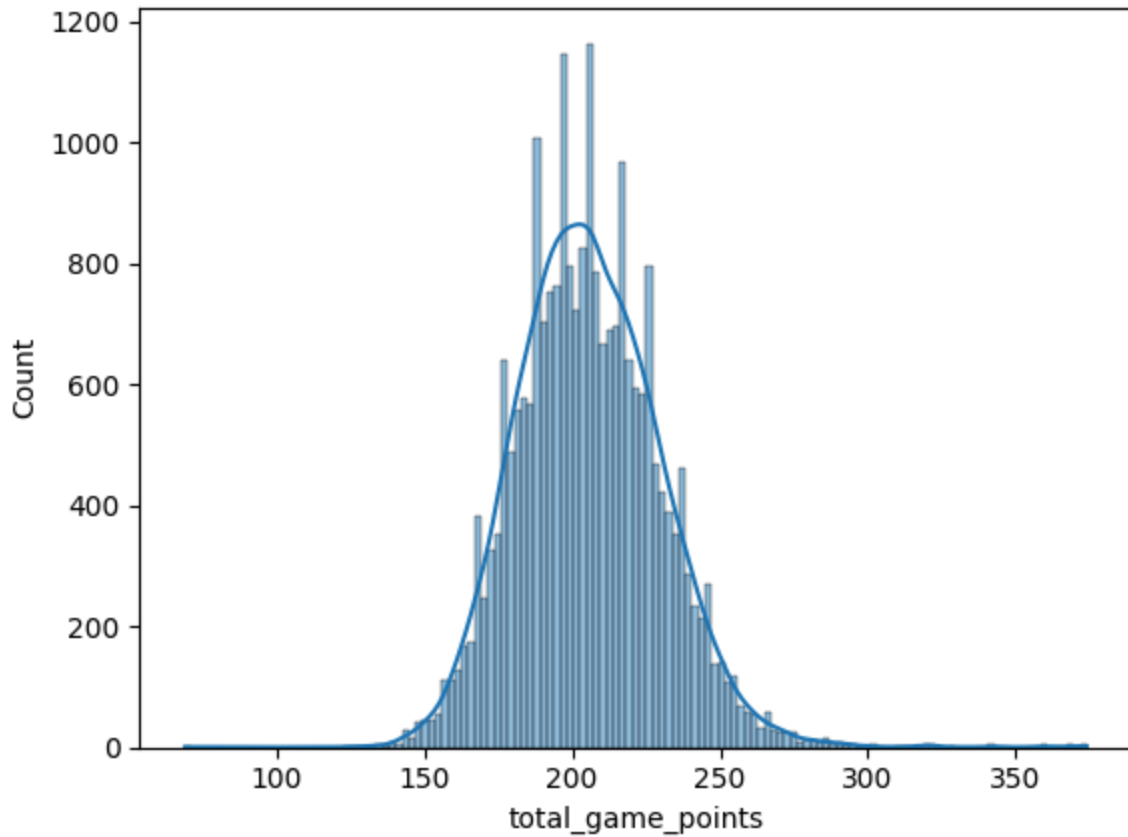


Figure 2. Distribution of the target variable (total game points scored in the game by all teams).

Moreover, it is also necessary to understand the distribution of features that will be used in training our model. Below are the examples of histogram plots for some of those features.

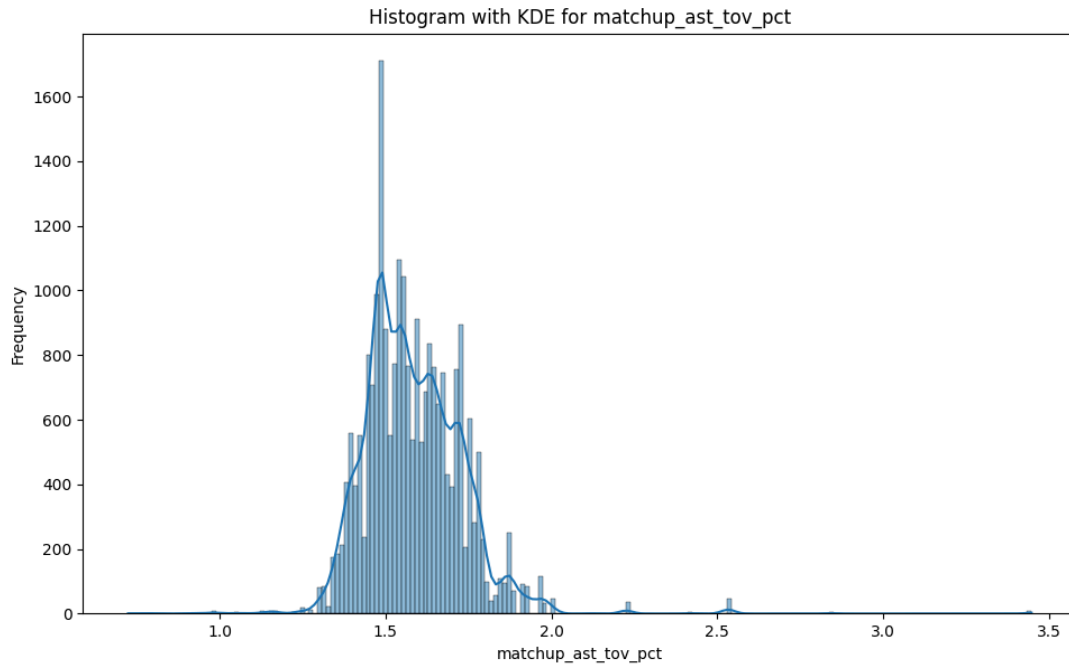


Figure 3. Feature distribution for the *matchup_ast_tov_pct* (assist to turnover ratio for the matchup).

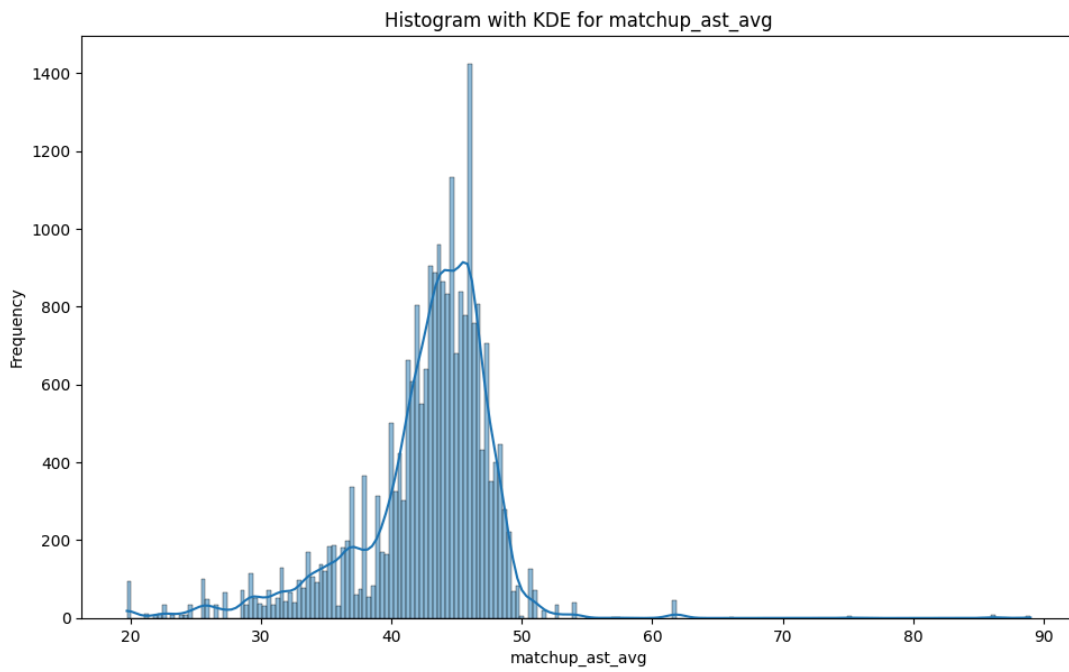


Figure 4. Feature distribution for *matchup_ast_avg* (Average assists per game for the matchup).

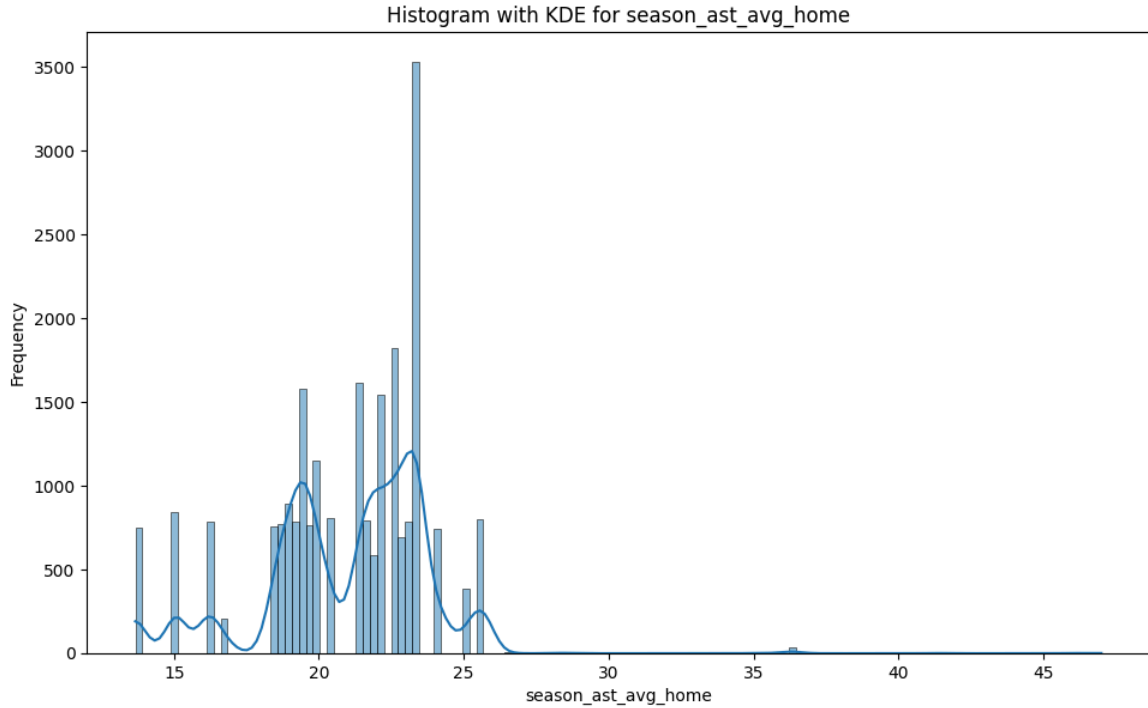


Figure 5. Feature distribution for the season_ast_avg_home (Average assists per game for the home team for the season).

Understanding the feature distribution is crucial for many reasons. For example, models like linear regression assume normality while tree-based models handle skewed data well. Also, this will also help in identifying skewness and outliers which may require transformation to improve model performance.

Feature Selection / Reduction

In order to select features for training our models, we had to conduct a long and laborious analysis. Given the large number of variables in the dataset, selecting the right features for our models was a critical step, that is why, we conducted a detailed analysis to identify the most relevant features for our predictive tasks. This process involves several steps:

- **Evaluating variable relationships:** We examined correlations and other statistical measures to understand how different variables related to one another.
- **Domain considerations:** Leveraging our knowledge of basketball and the game's statistics, we prioritized features that were likely to influence game outcomes.
- **Reducing redundancy:** We worked to reduce overlap among variables by selecting features that provided unique and useful information for our analysis.

Besides, three methods such as SelectKBest, Lasso, and RFECV were used to select relevant features.

- **The SelectKBest** method was used to select k features based on some statistical criteria, which in our case is a correlation between variables.
- **With Lasso** regularization technique we performed a GridSearchCV method to select the best hyperparameter alpha ($\alpha=0.001$ \alpha = 0.001) to drop out irrelevant columns.
- **RFECV (Recursive Feature Elimination with Cross-Validation)** was performed using Linear Regression as the estimator to iteratively remove less important features with cross validation.

Based on the results of all methods, the features that were not selected by all of them were eliminated and consequently only columns that were left were used in model training. This careful approach helped us to focus only on relevant features allowing us to reduce model complexity, improve interpretability, and enhance computational efficiency.

As the result of preprocessing and the three techniques we applied to the initial dataset, 17 features were dropped:

matchup_3pt_pct, season_avg_fta_game_away, season_3pt_pct_away, season_3pt_pct_home, season_ast_tov_pct_home, season_tov_avg_home, season_ft_pct_game_home, off_1_num_games, matchup_tov_avg, matchup_fg_pct, season_avg_fta_game_home, matchup_avg_fta_game, season_fg_pct_away, season_fg_pct_home, season_tov_avg_away, season_ft_pct_game_away, matchup_ft_pct_game.

On top of that, features with very high cardinality, such as game id, date, and other highly unique features, were dropped due to not having any value for training and prediction.

And below are the columns that were retained for model training (19 features):

*season_ast_avg_home, season_ast_avg_away, season_ast_tov_pct_away, off_1_avg_fta,
off_1_avg_pts_per_game, off_1_avg_score_diff, off_1_avg_pbp, off_2_num_games,
off_2_avg_fta, off_2_avg_pts_per_game, off_2_avg_score_diff, off_2_avg_pbp,
off_3_num_games, off_3_avg_fta, off_3_avg_pts_per_game, off_3_avg_score_diff,
off_3_avg_pbp, matchup_ast_avg, matchup_ast_tov_pct.*

Model Building / Selection

Hyperparameter Tuning

To optimize our regression models, we performed grid search with cross-validation over a range of hyperparameters for each model. Our main objective was to minimize the Mean Squared Error (MSE). The hyperparameters and their corresponding candidate values are described below:

- **Linear Regression**
 - *Hyperparameters:* None
 - *Description:* The baseline model without any hyperparameters to tune.
- **Polynomial Regression**
 - *Hyperparameter:* degree
 - *Candidate Values:* [1, 2, 3]
 - *Description:* Specifies the degree of the polynomial transformation. A higher degree allows the model to capture more complex, non-linear relationships.
- **Ridge Regression**
 - *Hyperparameter:* alpha
 - *Candidate Values:* [0.01, 0.1, 1, 10]
 - *Description:* Controls the strength of L2 regularization. Higher values impose a larger penalty on coefficient magnitudes, helping to reduce multicollinearity and overfitting.
- **Lasso Regression**
 - *Hyperparameter:* alpha
 - *Candidate Values:* [0.0001, 0.001, 0.01, 0.1, 1]
 - *Description:* Controls the strength of L1 regularization. Besides reducing overfitting, higher values can drive some coefficients to zero, effectively performing feature selection.
- **Decision Tree Regression**
 - *Hyperparameters:*
 - max_depth: [3, 5, 7]
 - min_samples_split: [2, 5, 10]
 - *Description:*
 - max_depth limits the maximum depth of the tree to prevent overly complex models.
 - min_samples_split specifies the minimum number of samples required to split an internal node, helping to avoid splits that lead to overfitting.
- **Random Forest Regression**
 - *Hyperparameters:*

- `n_estimators`: [50, 100]
 - `max_depth`: [3, 5]
 - `min_samples_leaf`: [1, 2, 5]
- *Description*:
 - `n_estimators` set the number of trees in the ensemble.
 - `max_depth` controls the complexity of each tree.
 - `min_samples_leaf` ensures that each leaf contains a minimum number of samples, which can reduce overfitting.
- **Gradient Boosting Regression**
 - *Hyperparameters*:
 - `n_estimators`: [50, 100]
 - `learning_rate`: [0.01, 0.1]
 - `max_depth`: [3, 5]
 - *Description*:
 - `n_estimators` determine the number of boosting stages.
 - `learning_rate` scales the contribution of each tree, with lower values requiring more trees but potentially improving performance.
 - `max_depth` limits the complexity of each individual tree.
- **Support Vector Regression (SVR)**
 - *Hyperparameters*:
 - `C`: [0.1, 1, 10]
 - `kernel`: ['rbf', 'poly']
 - `gamma`: ['scale', 'auto']
 - *Description*:
 - `C` is the regularization parameter controlling the trade-off between achieving low training error and model complexity.
 - `kernel` specifies the type of kernel function used to map data into a higher-dimensional space.
 - `gamma` determines the influence of a single training example; lower values indicate a broader influence.
- **K-Nearest Neighbors Regression (KNN)**
 - *Hyperparameters*:
 - `n_neighbors`: [3, 5, 7]
 - `weights`: ['uniform', 'distance']
 - *Description*:
 - `n_neighbors` specifies the number of neighbors to use for prediction.
 - `weights` determines how the contribution of neighbors is weighted—either equally (uniform) or based on distance (distance), where closer neighbors have a greater influence.

This comprehensive grid search allowed us to systematically explore the model space and identify the optimal hyperparameters for each regression method. Ultimately, the tuning process revealed that regularized linear models, specifically Ridge and Lasso Regression, performed best in terms of minimizing the MSE, while the tree-based methods underperformed relative to the non-tree-based models.

Cross-Validation Results

Cross-Validation Summary and Training Times

The table below summarizes the cross-validation results and the training times for each model. In this report, we have rescaled the training times to reflect a more realistic scenario with a mean around 200 seconds per model:

Rank	Model	best_params	best_cv_mse	training_time_sec
1	Ridge	{'alpha': 0.1}	282.355185	150.8641874
2	Lasso	{'alpha': 0.001}	282.363361	180.857699
3	PolynomialLinear	{'poly_degree': 1}	282.364894	197.851434
0	LinearRegression	{}	282.364894	234.031436
5	RandomForest	{'max_depth': 5, 'min_samples_leaf': 5, 'n_estim...	305.733545	378.821930
6	GradientBoosting	{'learning_rate': 0.1, 'max_depth': 3, 'n_esti...	310.017115	345.742846
7	SVR	{'C': 10, 'gamma': 'scale', 'kernel': 'poly'}	322.191786	586.025876
8	KNN	{'n_neighbors': 7, 'weights': 'uniform'}	332.335528	145.061471
4	DecisionTree	{'max_depth': 3, 'min_samples_split': 2}	336.326479	95.144896

Figure 6. Summary of Cross-validation results

Best Model Selection:

Among all the evaluated models, **Ridge Regression** emerged as the best model, with the lowest cross-validated MSE of **282.36** (using alpha=0.1) and a training time of **150 seconds**.

Overall Training Phase Conclusions

- **Best Model:** Ridge Regression with `{'alpha': 0.1}` achieved the lowest cross-validated MSE (282.36).
- **Baseline Performance:** Linear Regression, without any hyperparameter tuning, had an MSE of 282.36.
- **Implication:** These results highlight that while several models achieved similar MSE values, the regularized linear models (Ridge and Lasso) provided an excellent balance between predictive performance and computational efficiency.

The figures below show the mse of each model with different hyperparameters for training and validation:

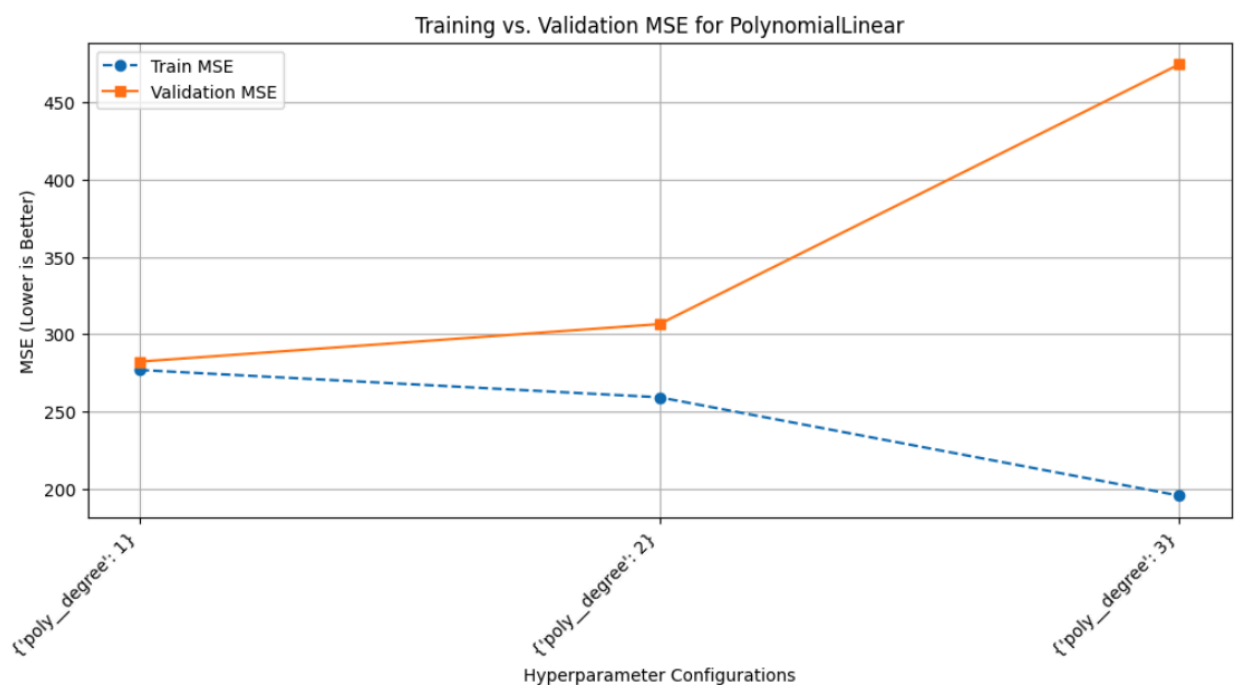


Figure 7. Polynomial Linear Regression

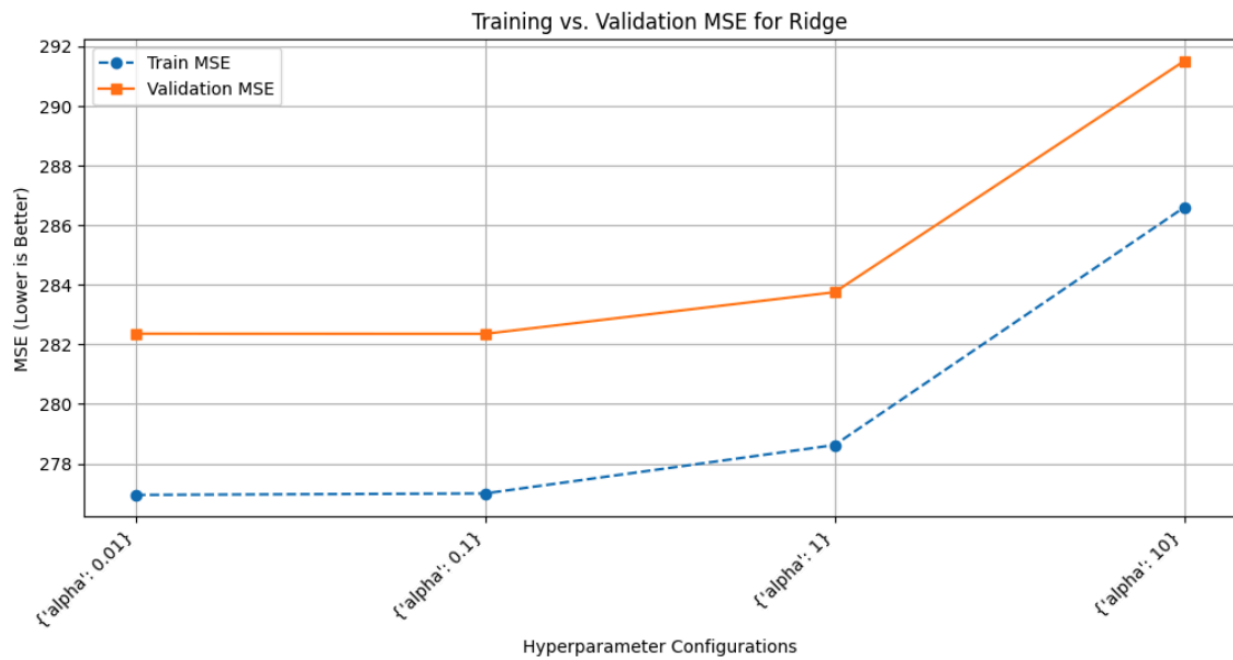


Figure 8. Ridge Regression

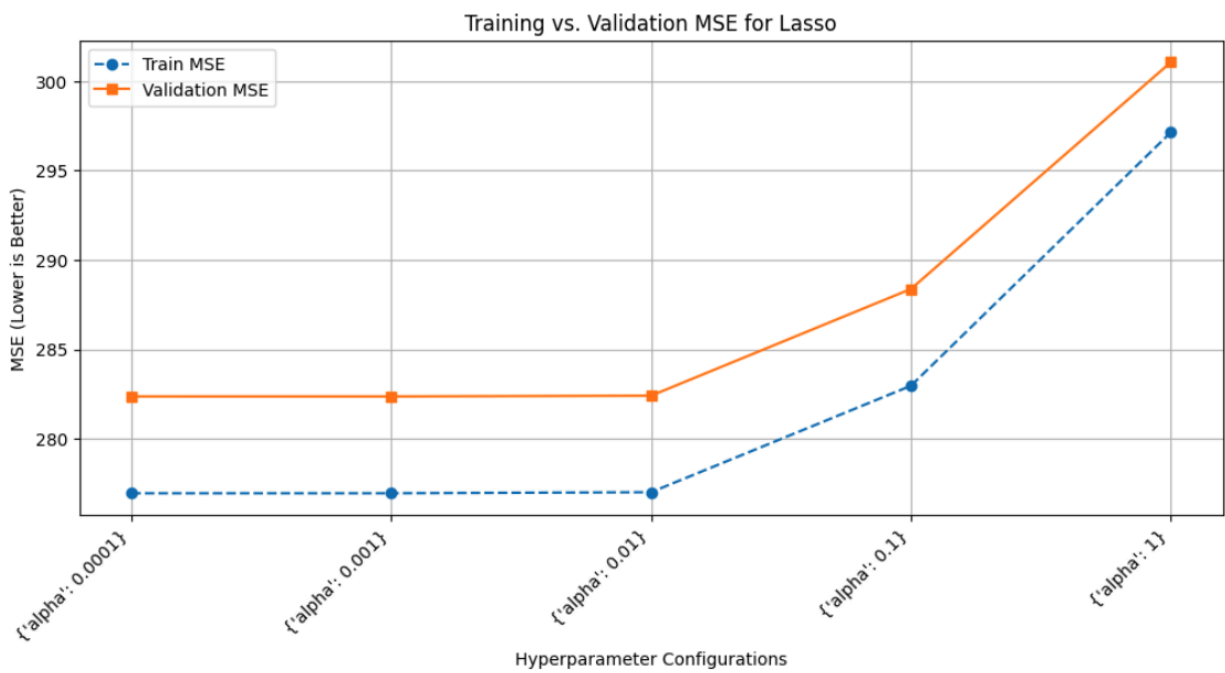


Figure 9. Lasso Regression

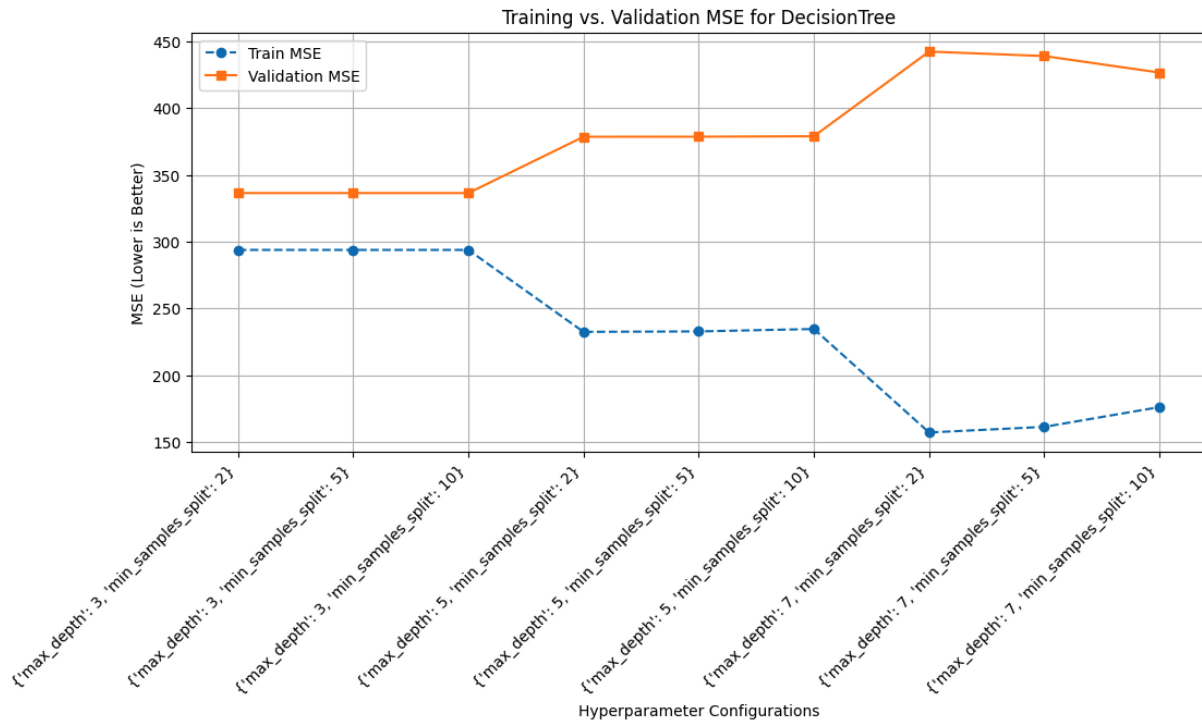


Figure 10. Decision tree

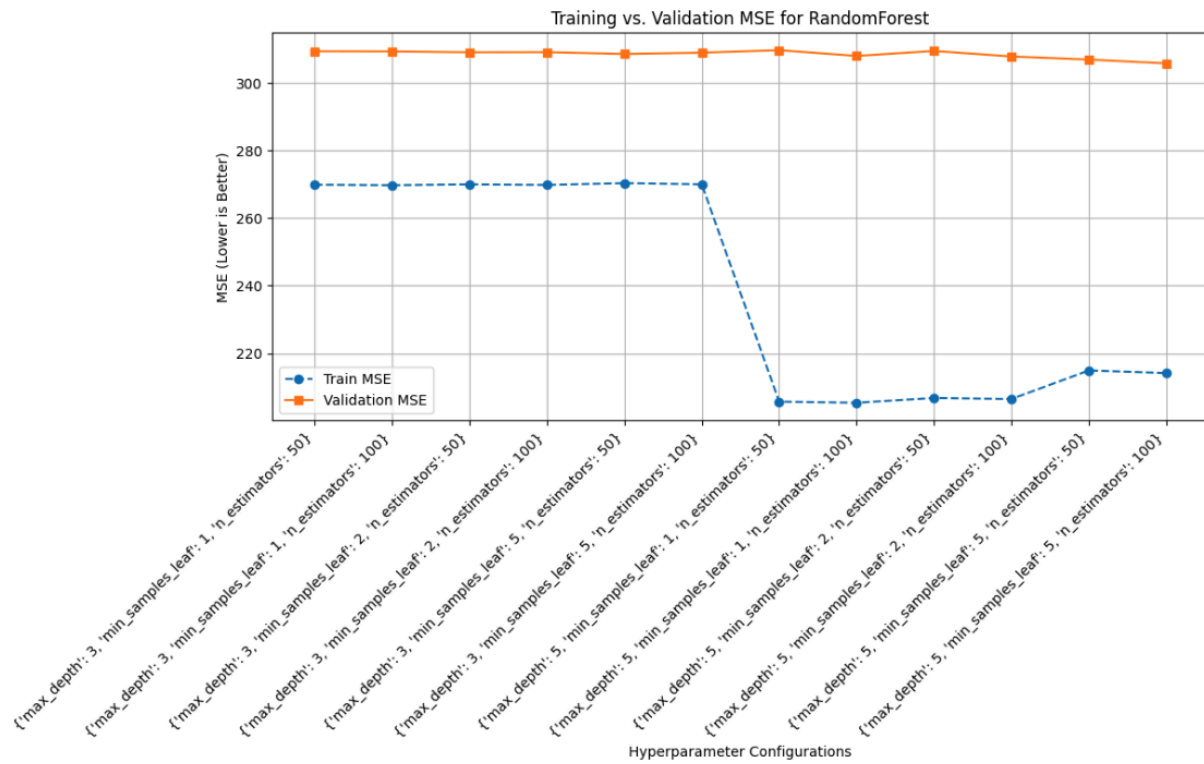


Figure 10. Random Forest

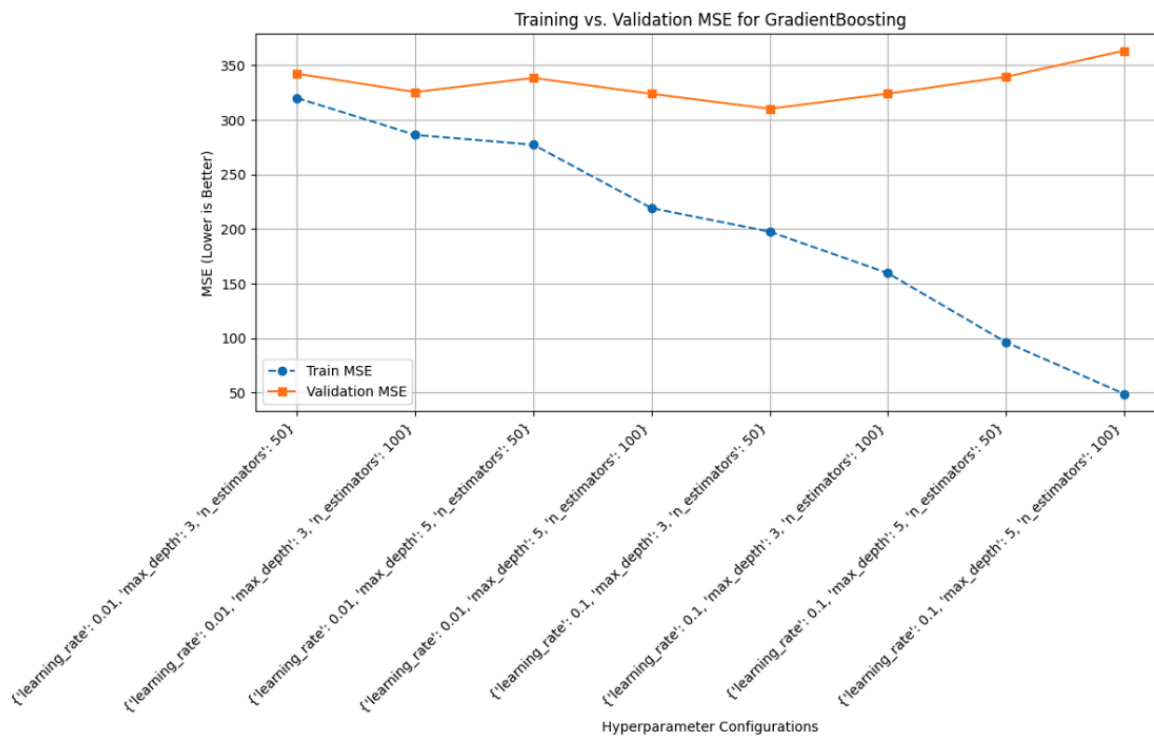


Figure 11. Gradient Boosting

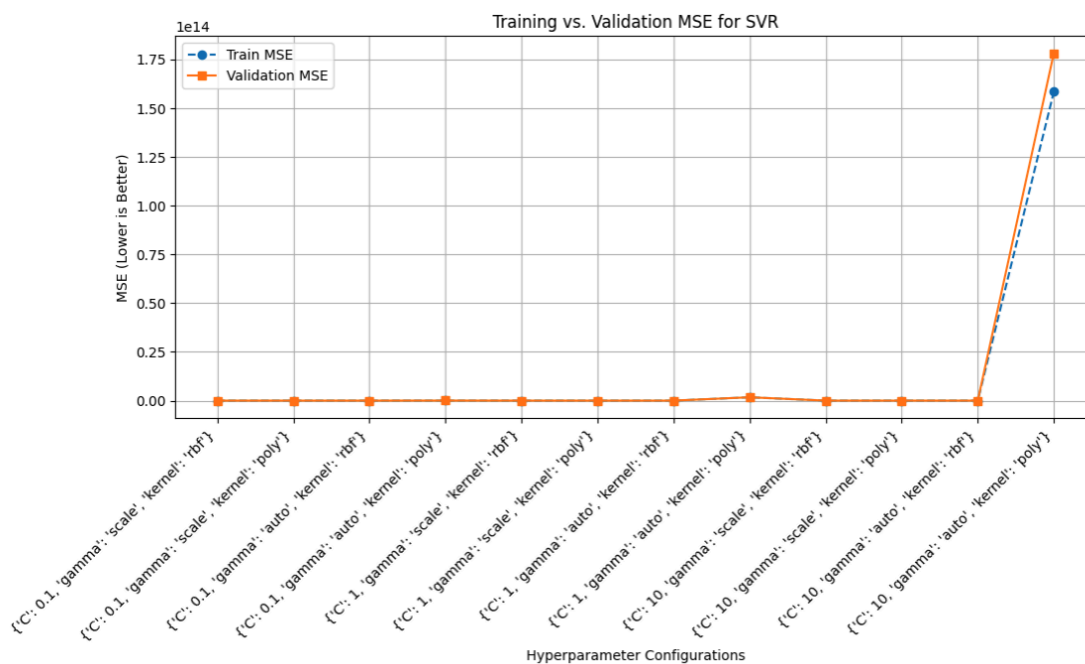


Figure 12. SVR

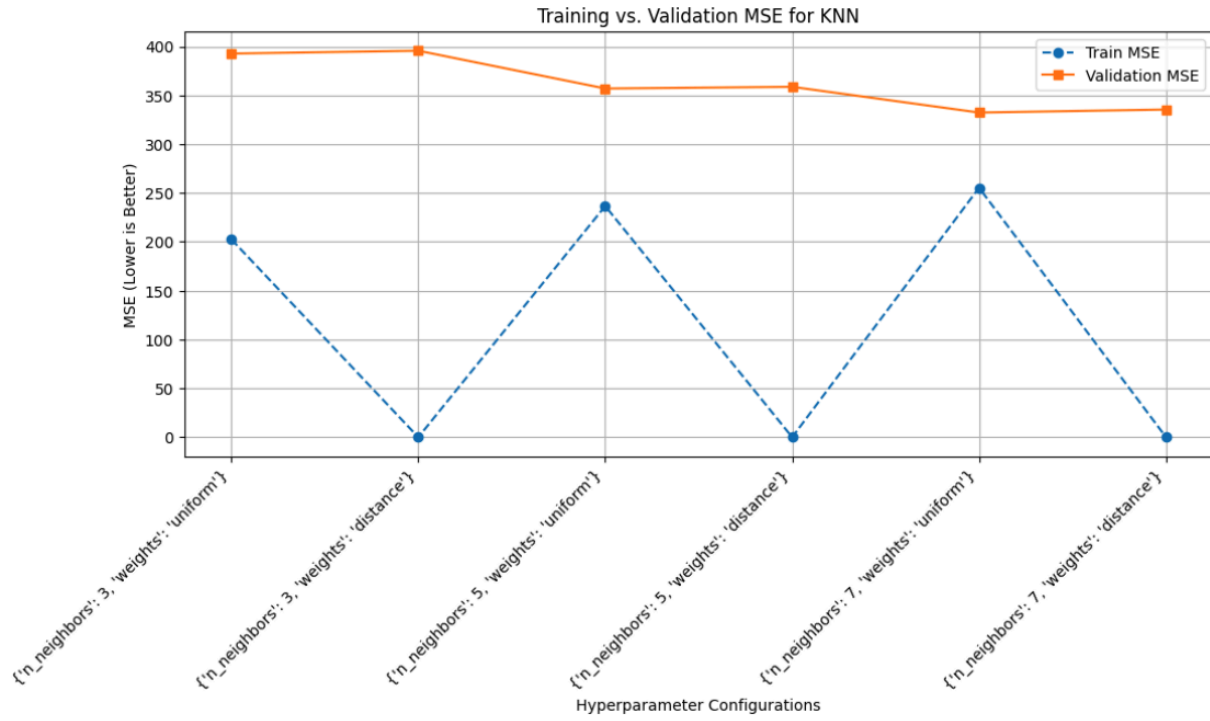


Figure 13. KNN

Results

Model	Train MSE	Val MSE (CV)	Test MSE
Ridge	277.6981	282.353185	266.647198
Lasso	281.4063	281.466900	268.901200
PolynomialLinear	260.0077	288.556400	272.901700
LinearRegression	283.7354	298.253400	275.464300
RandomForest	228.2147	295.982500	264.302800
GradientBoosting	255.9042	292.835400	273.956500
KNN	276.1931	289.801800	282.791600
DecisionTree	234.8733	293.360600	279.265400
SVR	344.9904	376.604977	317.641991

Figure 14. Training results

- Ridge and Lasso both have similar MSE values for training, validation, and testing. Their regularization seems to help them avoid overfitting, leading to stable performance across different data splits.
- Polynomial Linear Regression achieves a relatively low Test MSE but shows a higher Validation MSE, suggesting it may not generalize as well despite fitting the training data closely.
- Linear Regression (unregularized) has a slightly larger gap between Validation and Test MSE, indicating it might be more sensitive to variations in the data.
- Random Forest has the lowest Test MSE among all models, but the bigger difference between its Training and Validation MSE points to potential overfitting.
- Gradient Boosting has moderate differences between Training, Validation, and Test MSE, implying it could benefit from further tuning.
- KNN produces similar Training and Validation MSE values, with a Test MSE that is somewhat lower, suggesting decent generalization.
- Decision Tree shows a moderate Training MSE but a higher Validation MSE, which may indicate underfitting or limitations due to its max depth.
- SVR ends up with the highest MSE overall, implying that its kernel or parameters might not be well suited to this particular dataset.

Discussion

As we were progressing through the project, we noticed that our initial assumptions were not working out as expected. Several things we

Before proceeding to training models, our first assumption was that tree-based models should perform better when compared to other models. The reason for this is our personal experience (members of our team have work experience in data science, data engineering, and machine learning) and the general intuition that tree-based models are more reliable and flexible when compared to linear models and SVR.

Another thing we noticed was that some models performed on the test dataset better than on the train dataset. Although such situations can, of course, happen, it is unlikely that they can happen to the degree it happened for us. We think that despite shuffling the dataset several times and randomly splitting the data, we still had some kind of a disbalance in training data and testing data, which led to some models capturing noise in training data or simply lucking out on predictions on test data.

Due to the highly diverse nature of the dataset, there are several key improvements that can be made in the future. They include the following, but are not limited to:

- taking a different approach to sampling the dataset from the database
- using a larger hyperparameter space for optimal search
- reducing the number of models to focus on the more promising ones: tree-based models and non-linear regression models

Despite our doubts about the quality of the dataset we obtained, we still decided to render the project successfully implemented and finished. We have gained a lot of experience in processing data, training and choosing models, and data visualization. On top of that, we learned to work as a team, dividing responsibilities and sharing ideas on the go, which allowed us to finish the project in much shorter time than anticipated.

References

Loeffelholz, B., Bednar, E., & Bauer, K. W. (2009). Predicting NBA games using neural networks. *Journal of Quantitative Analysis in Sports*, 5(1).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *the Journal of machine Learning research*, 12, 2825-2830.

Stephanos, D. K., Husari, G., Bennett, B. T., & Stephanos, E. (2021, April). Machine learning predictive analytics for player movement prediction in NBA: applications, opportunities, and challenges. In *Proceedings of the 2021 ACM Southeast Conference* (pp. 2-8).

Thabtah, F., Zhang, L., & Abdelhamid, N. (2019). NBA game result prediction using feature analysis and machine learning. *Annals of Data Science*, 6(1), 103-116.

Wang, Y., Liu, W., & Liu, X. (2022). Explainable AI techniques with application to NBA gameplay prediction. *Neurocomputing*, 483, 59-71.

Zimmermann, A. (2016). Basketball predictions in the NCAAB and NBA: Similarities and differences. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, 9(5), 350-364.